Scale Language Specification

Status: alpha, living document. This specification is normative for the target language and records, per feature, what the current implementation does. A feature marked **implemented** is enforced by the test suite. A feature marked **partial** is specified but has named refusals or untested corners. A feature marked **planned** is part of the target and has no implementation yet. A feature marked **refused** is deliberately outside the current language and has a named refusal. The specification ships as one document; the topic documents remain the design rationale and the deep dives cited per chapter.

Conformance language. The words MUST, MUST NOT, SHOULD, SHOULD NOT, and MAY are used in the RFC 2119 sense. A MUST that the implementation does not yet meet is a **partial** status, not a licence to ignore it.

1. Scope and conformance

1.1 The language

Scale is a set-theoretic, concatenative language whose durable values live in an append-only image and whose execution state is never serialized. It has three front ends over one semantic core:

- the **core language**: the concatenative token surface plus the general IR (expressions, quotations, patterns, packages), used by direct programmers;
- **Prose sentence grammar**: a controlled English that compiles into the core language;
- the **prose surface**: a sentence-shaped program front-end that lowers to the general expression IR and runs on the same stack (D142–D149).

The core language is Scale's own surface, not the dropped unified Julia/Elixir language (D151). The Prose sentence grammar is a front end, not a second semantics. The prose surface is a front-end too: it adds no second evaluator. The current reference implementation is the core implementation; it is reused, extended, or replaced per component.

Different implementations are separate **implementation instances** in their respective host languages. They do not share a source language; they integrate through the actor model's IPC (11.7, D151).

1.2 The three artifacts

Scale ships three artifacts that share one version number:

- the **kernel**: the abstract machine and the C ABI;
- the **Prelude**: the language semantics, shipped as the F# reference today and as an image-resident program in the port;
- the **host adapters**: marshalling between a host and the kernel ABI.

The kernel has no Prelude state and no Prelude logic (invariants 3 and 4). The Prelude is swappable data (invariant 2). Removing or adding a Prelude feature must not require a kernel change (invariants 5 and 6).

Implementation layout (D137–D149). The kernel is `components/kernel`. The language is composed from `components/core` (runtime and image types, the pattern/expression/package ASTs, the vocabulary, the parser, the prose parser, and persistence), `components/typing` (the domain AST, lattice, SMT, violations, presentation, and emitter), `components/prelude` (the evaluator, runtime, scheduler, and writer), `components/verification` (the refiner, discourse, diagnostics, and decision tables), `components/demo` (the generated samples, player domain, fixed-specification harness, and showcase), `components/cli` (argument parsing, including the `.prose` mode), `components/shell-`

`host` (the process effect boundary and the prose script runner, D183), and `components/book-host` (the `.book` loader and the prose `cast` adapter, D184). `development/scale` holds the example, granular, property, and differential suite. `bases/scale-cli` composes everything and provides `Entry.runProseFile`, which delegates to `ProseShell.runFile`; `projects/scale` is the AOT publish target. The Nu engine is the external `components/nu-engine` node; `components/nu-host`, `bases/nu-game`, and `projects/game` are the host skeleton. The repository is colocated with Jujutsu; the sparse `Scale-code` workspace excludes `components/nu-engine` so the F# language server sees only Scale projects, while the main workspace remains the full Nu checkout (D182). Only the ABI and the image cross the host boundary; `just poly-check` enforces the graph. The canonical vocabulary is in `docs/irs.typ`: an IR is an abstraction, `TermIr` is `SymbolicAbstraction`, `LoweredInstructions` is the instruction set of `LoweredAbstraction`, the `-Ast` modules are `-Logic`, and `TypeInference` is `TypeRetrieval`. The other code names remain until a rename is scheduled (D185); the lowered instruction set was renamed (D190).

1.3 Normative sources

The order of authority is:

1. `docs/INVARIANTS.typ`, the architecture constitution;
2. this specification for language behavior;
3. `docs/decision-records.typ` for the rationale and the historical decisions;
4. the tests, which are the executable evidence for the status matrix;
5. the topic documents cited in each chapter.

Where the reference implementation and this specification disagree, the disagreement is a defect in one of them and MUST be resolved in the same change, not papered over.

1.4 Status legend

Status	Meaning
Implemented	Specified and enforced by the current test suite.
Partial	Specified; some cases are named refusals or are not yet tested.
Planned	Part of the target; no implementation yet.
Refused	Deliberately outside the current language; the refusal is named.
Extension	Part of the language but not on the kernel/Prelude port path.

2. Lexical structure

2.1 Program text

This chapter describes the **core** token surface. A program is a sequence of whitespace-separated tokens. There is no semicolon, no significant newline, and no comment form: the language is word-oriented, every word stands on its own with its own semantics, and a source reads like regular language (D131). Whitespace is a separator, never part of a word.

The prose surface has its own lexical rules (17.7): punctuation, lowercase keywords, text with spaces, and names canonicalized through the vocabulary.

2.2 Token classes

Form	Meaning	Status
`42`, `-1`, `3.5`	a real number, parsed as a 64-bit float	Implemented

Form	Meaning	Status
`"text"`	a single-word text literal; the body is the characters between the quotes	Implemented
`:name`	an atom literal; the reference to a named atom	Implemented
`?name`	a unification variable or pattern variable	Implemented
`^?name`	a pinned pattern variable	Implemented
`!?name`	a shadowing pattern variable	Implemented
`_`	a wildcard pattern	Implemented
`{`, `}`	a quotation, collected at run time	Implemented
`[`, `]`	an extension set literal	Implemented
any other token	a word call	Implemented

In the core surface a text literal is one whitespace-free word: whitespace is the word separator, so a quoted string containing spaces is not a single token (D131). The prose surface admits text with spaces and keeps its case (17.7).

2.3 Reserved and structural tokens

The following are syntax, not names, and are never shadowed by a lexical binding: `{`, `}`, `[`, `]`, numbers, text literals, `:name`, `?name`, and the internal return tokens (a leading NUL). Every other token is a name and may be shadowed by a lexical binding (D122).

2.4 Source positions

Every executed token carries a location, baseline, and position. A diagnostic reports them with the error. The character offset for a source violation is recorded when a source form is available (D76).

3. Surface syntax and evaluation shape

3.1 Concatenative core

Evaluation is stack-based. A token that is not syntax is a word call. Words pop their operands from the stack and push their results. There is no implicit argument order beyond the stack: the last pushed value is the top.

The core words are the arithmetic and comparison words (`add`, `subtract`, `multiply`, `divide`, `lessThan`, `greaterThan`, `equals`), the stack words (`duplicate`, `swap`, `drop`, `over`, `rotate`), and the control words below. The canonical spelling is camelCase for the comparison words; the lowered IR also accepts `lessThan` and `greaterThan` as the token names (D124).

These are the core-surface spellings. The prose surface writes lowercase words with spaces and canonicalizes them through the vocabulary (`less than` becomes `lessThan`) (17.7).

3.2 Quotations

`{` collects tokens up to the matching `}` into a text quotation and pushes its address. A quotation is data: it is not executed until `apply` or a control word runs it. Nested quotations are collected by depth.

```
5 { duplicate } apply add     // 10
```

3.3 `if`

The surface is `condition { then } { else } if`. `if` pops the else quotation, the then quotation, and the condition scalar, then splices the chosen quotation into the program. A zero condition

chooses the else branch; a non-zero condition chooses the then branch. The condition MUST be a scalar.

3.4 `apply`

`apply` pops a quotation address or a closure and runs it. A bare quotation runs in the caller's current frame (dynamic scope). A closure runs in its captured frame and restores the caller's frame through the internal return token. An `apply` with no target is a stack underflow, not a silent no-op.

3.5 `define` and `control`

`define` is the global word definition. The stack form `body "name" define` stores the body quotation under the name in the user-word table. The definition words read the name and quotation body from the program:

```
logic name { body }
cycle name { body }
cycle first { body } and second { body }
```

`logic` defines a non-recursive word. Its own name is not in scope in its body, so a self-reference is a named refusal. `cycle` defines a recursive word; the body uses the plain word name, with no separate `recurse` word. `and` groups `cycle` definitions: every group name is in scope in every group body, and the group is stored together. A later definition under the same name wins. `define` remains available as the stack form.

`control` is the form declaration: `handler "name" control` captures the current frame and installs a code cell as the implementation of the form. A later declaration under the same name wins. The default control forms (`{`, `if`, `apply`, `define`, `logic`, `cycle`) are seeded as image-resident words whose implementations name the matching kernel primitive.

3.6 `let` and `lambda`

`value "name" let` pops the name and the value, creates a child frame containing the binding, and makes that frame current. It produces no value. The binding is visible to a bare token (D122) and to `lookup`; `lookup` walks the frame chain and then the global binding table.

`{ body } lambda` captures the current frame in an image-resident closure and pushes it. A closure applied with `apply` runs its body in the captured frame and then restores the caller's frame.

3.7 Word resolution order

For a bare token that is not syntax, resolution is:

1. a word metaobject or control form (D21, D51);
2. the nearest lexical frame binding (D122);
3. a built-in named word;
4. a generic function (D19);
5. a user word (`define`);
6. the vocabulary;
7. otherwise `Unresolved(Word, token)`.

A lexical binding therefore shadows a built-in word, but syntax and literals are never shadowed. An inactive word is `InactiveWord`, never a silent fallthrough.

3.8 Failures

A failure is a value, not an exception, in the interpreter: every word returns `Result<EvaluationState, EvaluationError>`, and the main loop attaches source position and returns a `Diagnostic`. A named reason is matchable; an untyped string refusal is not part of the

language. The compiled path converts a runtime failure at the harness boundary into a named fault and the caller maps it back to a diagnostic.

3.9 History, bookmarks, and rollback

Event-stream recording is opt-in and off by default. `history-on` enables it and `history-off` disables it; a bookmark records its own step explicitly. While recording is on, every executed token appends a step recording the token, the stack before it, the program to resume, the location/baseline/position, and the previous step. The steps form a persistent chain, so appending is O(1) and `history`/`rollback` walk the chain. `history` reports the steps, `rollback` restores one, and a bookmark is a named step. These are interpreter features. Compiled artifacts exclude history, bookmarks, and rollback (D120): the lowered instruction set has no step record and no resumable program counter.

3.10 The F#-equivalent pipeline

The pipeline operator, `|>`, is the F#-equivalent pipeline. It exists for a 1:1 port of F# code, not for the concatenative core. It is left-associative and binds looser than a call: `x |> f(a) |> g(b)` means `(x |> f(a)) |> g(b)`.

F# passes the piped value as the **last** argument. The elaborations are:

1. `x |> f(a)` becomes `f(a, x)`.
2. `x |> f` becomes `f(x)`.
3. `x |> f(a) |> g(b)` becomes `g(b, f(a, x))`.

The two argument positions are independent of any language: `LastArgument` is the default, and `FirstArgument` is available for surfaces that use it. F# is the compatibility case for `LastArgument`; Elixir is the compatibility case for `FirstArgument`. A `Pipeline` node records the style so a front-end or adapter can choose either without changing the surface or the kernel.

The surface parses a value, the operator, and a stage. A stage is a word, a call, or a word followed by whitespace-separated stage arguments: `x |> f(a)` and `x |> f a` both elaborate to `Call(f, [a; x])`. A pipeline without a stage is `ExpectedExpression`. A stage that is not callable is the normal call-target refusal at lowering, never a silent token splice.

Stage arguments are a pipeline-surface convenience, not general currying. Outside a pipeline, a bare application `f a` remains unavailable. The pipeline supplies the final argument; first-class functions and general partial application are separate work.

The `Pipeline` node is preserved in the portable expression document. Lowering desugars it to `Call`; the node keeps the source offset and the style for diagnostics and a later Elixir-style elaboration.

3.11 Explicit partial application

_ in a call argument is a hole. A call with at least one hole is a `Partial` expression, not a complete call:

```
subtract(_ 10)        a function of one argument
map(_ xs)             a function of one argument
f(_ a _)              a function of two arguments, in hole order
```

The supplied arguments are evaluated once, when the partial value is created. The holes bind when the partial value is applied. The result is an ordinary closure: `apply` runs it, and `x |> f(a)` remains a separate pipeline rule.

This is explicit partial application, not automatic currying. _ outside a call argument is a named word, not a hole. A bare `f a` outside a pipeline is still refused, and there is no implicit "supply the missing arguments" rule. A call with no holes is a complete `Call`.

The `Partial` node is preserved in the portable expression document. Lowering turns it into a closure whose parameters are the holes and whose body is a call to the target; the supplied arguments are bound in the enclosing frame so they are evaluated once. The target must be a word or a lambda in the current lowered subset; any other target is the normal call-target refusal.

3.12 The reverse pipe

The reverse pipe, `<|`, exists only to run F# packages that use it. It is **discouraged**: it reads against the dataflow, and the forward pipeline is the preferred surface.

Semantics are the reverse of `|>`:

1. `f <| x` means `f(x)`.
2. `f a <| x` means `f(a, x)`.
3. `f <| g <| x` is right-associative and means `f(g(x))`.

The expression is a `ReversePipeline` node, preserved in the portable document. Lowering appends the value to the left call, or wraps a plain target as a one-argument call. Supplied partial arguments already on the left keep their positions.

The parser gives `<|` the lowest precedence and right associativity; it is not a synonym for `|>` and does not use `LastArgument`/`FirstArgument`. Automatic currying remains out of scope.

3.13 Type elaboration (P4, core)

Type inference is optional elaboration, not runtime evaluation. P4 derives a type and an exact effect set from the expression using the internal word registry and the package-boundary FCS read-off, or returns a named diagnostic. There is no user-facing signature syntax; inference never observes a running value to choose a type.

P4a provides the host-side type core: type variables, primitives, sorts, functions with latent effects, schemes, substitutions, constraints, an occurs-check, generalization, instantiation, and named errors. P4b checks these expression forms against the registry:

1. literals, words, calls, lambdas, application, and partial application with argument holes;
2. `Let` binding nodes with sound generalization (the `logic` surface spelling is still a design);
3. `if` branches with a unified result type;
4. blocks and pipelines.

The internal primitive registry lives in `components/prelude/evaluator/signatures.fs`. It types `add`, `subtract`, `multiply`, `divide`, `lessThan`, `greaterThan`, and `equals` as implementation metadata, not source annotations. F# package bindings get their types from FCS read-off.

P4c adds a parallel expression source map, `InferenceDiagnostic` with a path and an optional `SourceRange`, and the opt-in `lowerExpressionWithInference` handoff. `parseLocated` supplies the root range and precise call/partial target and argument paths; other child paths are the next range slice. `scale --prose-typecheck PATH` parses, infers, and lowers without running. A type diagnostic stops lowering; the existing lowering refusal remains a separate layer. The canonical expression tree and portable document remain range-free.

P4 does not provide type rows, subtyping, type-level lambdas, dependent types, or pattern-match inference. An unknown primitive or expression is refused; ambiguous multiple dispatch is resolved from inferred arguments and is a diagnostic when undecidable. Effects are exact sets, not rows. Child source ranges and the `logic`/`function` surface remain open; the core and root located path are implemented, while the complete surface remains partial.

3.14 F# abstraction and manifest

`FSharpAbstraction` is a versioned in-memory subset for the Prelude and F# packages: a module name, version, source hash, and bindings. A binding carries a name, parameters, an expression body, the plain names it calls, a source range, and the F# recursion kind (`non-recursive`, `recursive`, or `mutually-recursive`). The abstraction is not a second runtime; it lowers to the same `PackageUnit`/`PackageUnitGroup` records used by books and `logic`/`cycle` words.

A manifest has schema version `1`, a package name, a mode (`automatic`, `mixed`, or `strict`), and per-binding declarations. A declaration kind is `non-recursive`, `recursive`, or `mutually-recursive` with a `mutual` name list. A `reason` is `uninitialized-value`, `cross-module`, `value-restriction`, or `other:`. In `strict` mode every binding must be declared; `mixed` accepts declarations for some bindings; `automatic` needs none.

An explicit declaration must agree with the F# kind. A disagreement without a reason is `ManifestDisagreement`; a disagreement with a reason is accepted and tracked as an override. The resulting classification record carries the binding name and `BindingKind`; it has no tier field. `FS0040`, the unused-recursive warning, and the value-restriction warning map to Scale diagnostic codes before they reach a user. Metadata crossing IPC is `(name, kind, version)` only; a function body is never serialized.

The product and AOT runtime do not link FCS. FCS is a build-time dependency. Its adapter emits a `ReadOffModule` with names, parameters, call names, source ranges, recursion kinds, and bodies for the supported expression subset: literals, names, calls, lambdas, `let`, `if`, sequential blocks, and tuple/list blocks. `fromReadOff` combines that record with a hand-supplied body when one is present. An unsupported F# expression or pattern is a named refusal, never a guessed translation. A schema mismatch is a named refusal.

4. Values and the image

4.1 Identity

An `EntryIdentity` is a distinct reference into the store lexicon. It is a struct wrapping an integer and is not a host integer in the language: it projects to a rank-0 tensor at a boundary. Identity equality is reference equality.

4.2 Tensors

The only element type of the value domain is a real number. A `Tensor` is a `Data` array plus a `Shape`. A scalar is a rank-0 tensor. Tensors are immutable: an operation produces a new tensor and a new store entry. Tensor order is significant; `Tensor.append` appends at the end, so the stack top is the last element.

4.3 Text and atoms

Text is stored as a sequence of character codes in a tensor and interned in the store's text cache. `TextRead` returns the interned text; `TextEquals` compares two text entries. Atoms are text entries reached through `:name`; they are distinct from text literals only by their surface form.

4.4 The store

The store owns the lexicon, the text cache, the next index, the association tables, and the named roots. It is append-only. An update writes a new vector and re-points a root; it never rewrites or truncates an entry. The association-vector primitives (`addToEnvironment`, `findPair`, root add/set) are the environment substrate.

The execution-root registry (`RootKind`/`rootKinds`) names `VocabularyTable`, `UserWordTable`, `BindingTable`, `SetTable`, `DeclarationTable`, `GenericFunctionTable`, `WordTable`, `HistoryTable`, `ClassTable`, `InstanceStateTable`, and `ChannelsRoot`. The current and saved frames are control state, not registry roots. Roots are execution state except where a feature explicitly makes them durable; a reader re-threads them. A new root adds one `RootKind` case and the coverage test fails until it is in `rootKinds` and `getRoot`/`setRoot`.

4.5 Durable data versus execution state

The image is the authority for durable and cross-actor values. Execution state is never serialized: the stack, the current and saved frames, the scheduler and actor state, ownership state, the event-stream history, handlers, and the program counter. Persistence serializes only durable or promoted entries; it does not serialize a stack or a frame.

4.6 Promotion and epochs

A value crossing an actor or persistence boundary is promoted to the image; it must never leave a dangling reference into the sender's epoch. Promotion reclassifies an entry as durable without moving it. Physical reclamation closes a closed epoch's unpromoted entries when no live root reaches them. There is no global mutable heap and no per-process tracing GC in this design.

5. Evaluation model

5.1 The loop

Evaluation is a token loop over a `Program` (a token list) and a stack. Each token decrements the reduction budget, increments the consumed count, and appends a history step. The loop is the single failure decision point: a word failure is offered to the condition system; a supervised child becomes an exit; an unsupervised actor aborts the run with a diagnostic.

5.2 Reductions and deadlines

`Reductions` is a fairness budget, not a failure. Exhausting it yields to the next ready actor and resumes with a fresh budget. A `Deadline` is an actor's token budget for its incarnation; exhaustion produces `TimeoutExit`, which the supervisor decides. Neither is `ReductionExhausted`.

5.3 Control and call frames

`CurrentFrame` is the active lexical frame. `CallFrames` is the saved caller chain. A closure call pushes the caller's frame and installs the captured frame; the internal return token restores it. A return without a call is a named refusal (`UnmatchedReturn`), never a silent no-op.

5.4 Actor steps

The scheduler owns the ready queue and the current actor. `spawn` creates a child; `yield` gives the CPU to the next ready actor; `halt` ends normally; `kill` ends with a reason. A supervised child's failure is an exit its supervisor consumes; an unsupervised actor aborts the run. See chapter 10.

5.5 Failure attribution

A failure MUST be attributed to the layer that raised it: a kernel/abstraction failure, a primitive failure, a Prelude word failure, and a parse failure are distinct. The kernel flattens a typed error to `(code, message)` only at the C ABI edge. The compiled harness raises a named fault and the caller maps it back to the same diagnostic category.

6. Binding and scope

6.1 Frames

A frame is an image-resident association vector with a parent pointer. `let` creates a child frame with one binding and makes it current. `lookup` walks the chain (depth limit 1024) and then the global `BindingTable`. A frame is data; the pointer to the current frame is execution state.

6.2 Bare names

A bare token resolves to the nearest lexical binding before it reaches the built-in word table (D122). This is a language rule, not a lowering detail: the interpreter and the lowered path MUST agree. A binding may therefore shadow a built-in word. Syntax and literals are reserved and are never shadowed.

6.3 `lookup`

`name lookup` pops a name and pushes the binding. It searches the current frame chain first, then the global binding table. An unbound name is `Unresolved(Binding, name)`, never a silent false.

6.4 Closures

`lambda` captures the current frame. A closure's body runs in the captured frame; the caller's frame is restored on return. A bare quotation applied with `apply` keeps the caller's frame (dynamic scope). The distinction is observable and is part of the specification.

6.5 Lowered slots

The lowered expression path represents a lexical binding twice: a frame binding (for `lookup`) and a fresh slot (for a bare name). A lexically bound `Word` lowers to `PushSlot`; `let` lowers value, name, `let`, `StoreSlot`. A lambda's parameters are slots in a named unit; the unit prologue copies its frame into slots. Capture is free-variable: only the enclosing names the body uses are captured, accounting for parameters, inner `let` bindings, pattern variables, and nested lambdas (D133).

6.6 Shadowing and duplicate names

A nested `let` shadows an outer binding; the nearest binding wins. Pattern variables are scoped to their clause and shadow an outer binding. A duplicate Fresh pattern variable in one clause is `DuplicatePatternVariable`; a duplicate lambda parameter is `DuplicateParameter`. Bindings named by syntax or a literal are refused with `ReservedBindingName`.

6.7 Calls to bound names

`Call(Word name, arguments)` lowers to a closure call when `name` is lexically bound: the arguments are evaluated, the bound slot is pushed, and `CallClosure` runs. The bound value must be a closure; a non-closure bound value is a named lowered failure. This supersedes the earlier `CallTargetBound` refusal (D177) and is what lets a `logic` function be called by name after its definition.

7. Pattern matching

7.1 Surface

```
subject { pattern -> body | pattern -> body } match
```

`match` pops the clause quotation and the subject. Clauses are tried in order. A clause that matches binds its variables and splices its body into the program. If no clause matches, the result is `NoClauseMatched(subject)`, a named failure.

7.2 Pattern IR

Pattern	Meaning	Status
`Wildcard`	_, matches anything, binds nothing	Implemented
`Variable(name, Fresh)`	first occurrence; binds the subject	Implemented
`Variable(name, Repeated)`	second occurrence; must unify with the first	Implemented
`Variable(name, Pin)`	`^?name`; resolves through the binding table	Implemented
`Variable(name, Shadowing)`	`!?name`; always rebinds	Implemented
`NumberLiteral`	scalar equality	Implemented
`TextLiteral`	text equality	Implemented
`AtomLiteral`	atom text equality	Implemented
`Constructor(name, children)`	constructor head and arity, then children	Implemented
`TypedPattern(sort, child)`	typed term, then child	Implemented

7.3 Ground encoding

The matcher reads the same term encoding the unifier uses:

- a compound is a vector `[-2; head; argument; ...]`;
- a typed term is a vector `[-3; sort; term]`;
- an atom is any text entry that is not a `?name` variable.

A `Constructor` with no children also matches an atom with the same text; a `Constructor` with children matches only a compound with the same head and arity.

7.4 Roles

`Fresh` binds on first occurrence. `Repeated` requires the name to be already seen in the clause and unifies with the bound value. `Pin` reads the binding table and unifies with it; a missing pin is `MissingPin`. `Shadowing` always rebinds, including over an existing binding. A `Repeated` name that was not seen is `UnboundRepeated`.

7.5 Guards

A guard is a named predicate with a pinned version: `subject { pattern when predicate -> body } match`. The predicate runs in a fresh reduction budget with an empty stack and an empty frame; a non-zero scalar is true, zero is false. A predicate failure is `GuardFailed(name, reason)`, never a silent false. The version is part of the portable contract; a guard whose pinned version is not available is a refusal.

7.6 Open subjects

If the subject is an open unification variable and the pattern is not a wildcard or variable, the matcher encodes the pattern as a term and unifies it with the subject, then collects the substitutions. An undecidable case is `PatternUndecided(reason)`, never a structural false. Shadowing on an open subject is undecided.

7.7 Analysis

`PatternAnalysis` provides best-effort reachability, redundancy, and closed-kind exhaustiveness. Best effort is explicitly not a proof: a pattern set is not claimed exhaustive unless the analysis says so, and an analysis failure is reported, not treated as a pass.

7.8 Lowered subset

The lowered expression path compiles ground patterns to inspection branches and slots: wildcards, fresh variables, number/text/atom literals, constructors, and typed patterns lower. `Pin`, `Repeated`, `Shadowing`, and guards are named refusals; an open subject remains the interpreter's job. The clause body sees its variables as lexical slots. See D125.

8. Sets, sorts, and unification

8.1 Extension sets

`[ ... ]` is an extension literal: the body is evaluated and the resulting values are admitted to a set. `[]` is the empty set. A value set is an image-resident structure with a latest-write-wins association vector; the roots are `SetsRoot`, `DeclarationsRoot`, and `GenericFunctionsRoot`.

8.2 Naming and sealing

`S :Name seal` seals a set under a name; `:Name` is a reference. A sealed set is nominal: equality reads the name. An unsealed set is structural: equality reads the extension. `unname` removes the name. A predicate seal and a constructor seal are the named-predicate and named-constructor forms.

8.3 Set operations

`and`, `or`, and `nor` are the lattice operations. `set-union`, `set-intersect`, `set-except`, `set-member`, `set-complement`, `same-shape`, `same-name`, `sort-of`, `merge`, `filter`, and `set-subset` are the set words. `is` is membership. `model` declares a domain; `admit` extends it. Membership deduplicates: a set has no multiplicity and no order.

8.4 Sorts

A sort is a pair `(N : option Name, X ⊆ U)`. Nominal equality reads `N`; structural equality reads `X` (extensionality). The operations are the dial between the two readings:

- narrowing adds or relies on `N`, making two structurally identical sets distinct;
- widening drops `N`, making two names for the same set the same;
- restricting or enlarging `X` moves the structural projection.

The set operations are the teeth of the dial. `set-subset` is the value-level subset operation; a type-level subtype relation is not a word until a consumer needs it (D131). Intersection, union, complement/top, bottom, and refinement types are implemented. Sequences and recursive types are target (D132); multisets and co-products are out until a consumer appears.

8.5 Unification

Unification is first-order, image-resident, and nominal where the term carries a name. A term is a variable (`?name`), an atom, a compound, a typed term, or a single-parameter binder. The substitution lives in `BindingTable`; the store is append-only, so unification writes new entries and re-points the table. The occurs check is enforced. `Undecided` is a named refusal, never a false. Higher-order unification accepts Miller patterns (D59).

8.6 Classification

`class-of` and the entry-kind lattice classify store entries (D32). A membership verdict is one value; an unknown kind is `UnknownKind`, not a silent extension.

8.7 Sequences

A sequence is a set with an order: a value plus a position function. The `Tensor` is already ordered, so the representation is the first widening past sets. The target adds an ordered-collection projector,

order-preserving operations, and ordered pattern support. Set operations do not apply directly: a union of sequences is concatenation, not set union. Sequences are planned (D132).

8.8 Recursive types

A recursive type is the least fixed point of a type constructor: a list, a tree, a recursive message, or the language's own expressions, patterns, classes, and packages. The type layer needs a recursive reference and a roll/unroll or fixpoint rule; the unifier already has an occurs check and the matcher handles recursive compounds at run time. Recursive types are planned (D132).

9. Classes, slots, and instances

9.1 The cell model

A class is an image-resident record with a name, a superclass chain, and slots. A slot has a name and a default. An instance is an image-resident record naming its class and holding slot values. All of it is data in the image; the current instance table is a root.

9.2 Words

`class`, `slot`, `create-instance`, `slot-value`, `set-slot-value`, `with-fields`, `class-precedence`, `subclass-of`, `instance-of`, `metaclass`, `use-metaclass`, and `cell-class` are the surface. `with-fields` binds a field in a body frame.

9.3 Dispatch

Multiple dispatch is nominal and image-resident: the generic function records its methods, and the most specific applicable method is selected from the argument classes' precedence order (D19, D47). The exact-specializer fast path is a cache, not compile-time resolution; it is guarded and falls back to the dispatcher.

Method combination supports `before-method`, `after-method`, `around-method`, `call-next-method`, and `next-method-p`. Metaclasses are the class of classes; `cell-class` exposes the cell class closure.

9.4 Status

Classes, slots, instances, precedence, nominal dispatch, combination, metaclasses, and closures are implemented (D43–D50). Linear access to instances is enforced through the ownership layer (chapter 12); compaction and reclaim use the same roots.

10. Conditions and restarts

10.1 Failures as values

A failure is a value with a named reason. Every word returns a `Result`; the loop decides whether the failure is handled, becomes a supervised exit, or aborts the run. An error is never swallowed into a false or an empty value.

10.2 Conditions and restarts

`condition name message` declares a condition. `restart name handler` declares a restart. `with-handler` installs a handler for a dynamic extent. `signal` raises a condition; `invoke-restart` transfers control to a restart. Handlers and restarts are frame-scoped and are execution state; a handler that does not handle the condition returns control to the search, not to a default.

10.3 Attribution

The condition system reports the raising layer and the rule violated. A handler failure, a missing restart, and a condition with no handler are distinct named refusals. The kernel's typed errors keep their family; only the ABI edge flattens them to `(code, message)`.

11. Actors, channels, and supervision

11.1 Actors

An actor is an image-resident record with a status, a parent, a handle, a mailbox, links, monitors, and a pending exit. The root actor is created with the initial state. `spawn` creates a child; `self` pushes the current handle; `yield` gives the CPU to the next ready actor; `halt` ends normally; `kill` ends with a reason. A handle is a private capability: it is not forgeable from an integer or a text.

11.2 Mailboxes and messages

`send-to` delivers to a handle; `receive-message` takes the next message (or blocks); `receive-matching` takes the first message matching a predicate. Reserved system messages are distinct from user messages (D75). A mailbox is a channel; the endpoint's open flag and queued messages are image state, and only the blocked-waiter set is host-side control state.

11.3 Links, exits, and monitors

`link` makes two actors' exits observable to each other; `unlink` removes it. `trap-exit` turns an exit signal into a message; `exit-signal?` and `exit-reason` inspect it. `monitor`/`demonitor` observe without linking; `down?`, `down-ref`, and `down-reason` read the monitor result. `register`/`unregister`/`whereis` are the named registry.

11.4 Supervision

`one-for-one`, `one-for-all`, and `rest-for-one` are the restart strategies. `supervise` installs a strategy. A child failure is an `ErrorExit`; a deadline exhaustion is a `TimeoutExit`; the supervisor decides whether the child restarts, the group restarts, or the failure escalates. An unsupervised actor's failure aborts the run.

11.5 Channels and ChannelEvents

`channel` creates a channel; `send`, `receive`, and `close` are the operations. `send-event`, `receive-event`, `choose`, `wrap`, `guard`, and `with-fallback` form ChannelEvents; `sync` is the blocking synchronization. A blocked receive is a distinct outcome, not an error.

11.6 Status

Actors, mailboxes, links, trapped exits, monitors, the registry, supervision, channels, and ChannelEvents are implemented (D20, D22, D63–D69, D75). BEAM operational parity (copy-on-send and a per-process tracing GC) and the unmanaged arena backend are optional. Implementation-instance IPC is a target track, planned (D151); it extends the distribution direction recorded in D133. The `scale_transport_send` ABI seam exists, but instance identity, discovery, remote send/receive, delivery, partition, and security are undesigned.

11.7 Implementation instances and IPC

An **implementation instance** is one running actor system in a host language. Different implementations do not share a source language, an AST, a type hierarchy, or a dispatch model. They integrate through the actor model: an actor handle may address an actor in another instance, and a message crosses the instance boundary through the host adapter's transport.

A remote send carries an envelope: the target handle, the sender handle, an optional reply channel, a portable payload, and a message identity. The envelope and body share one message; only bulk data may be out-of-line. A payload is a durable image entry or a canonical document; a host object, a closure, a frame, a stack, a tensor view, or execution state never crosses. A remote handle is a capability, not a forgeable name.

`send-to` recognizes a host-registered remote handle tensor `[RemoteHandleTag; instance; capability]`. A tagged tensor that the host did not register is not a capability. It records a `TransportRequest` in the scheduler and returns; the host drains the request after the run, reads the message entry, and performs the transport. The evaluator performs no I/O. The host delivers the numeric reply to the sender's mailbox and resumes the sender, so `send-to; receive-message` receives the remote result (D157, D158).

A failure is named: an unknown instance, an unreachable instance, a transport fault, and a remote exit are distinct. A partition is an outcome, not a silent loss. The kernel frames and checks the handle; the host adapter owns transport semantics (`scale_transport_send`, D109). The wire encoding is the Scale actor wire protocol, implemented in-repo and Cap'n Proto-compatible for the frozen subset (D152, D153); compatibility is gated by a reference conformance test. Schema tooling may take Protobuf's plugin contract and field-number metadata, but those stay off the wire. S1 is implemented (D154): one connection, one numeric envelope, and the root actor mailbox. S2 is implemented (D155): a capability names a non-root worker, a monitor reports its normal exit, and an unreachable instance is a named failure. S2.5 is implemented (D157, D158): `send-to` accepts a remote handle and records a host transport request; the host delivers the numeric reply back into the sender's mailbox. The host test sends 21, receives 42, receives `down normal`, and the language receives 42 on the stack. Text replies, down delivery into the language, discovery, authentication, remote code change, and partition semantics remain planned (D151–D153); `docs/actor-ipc.typ` records the model and staging, and `docs/binary-format.typ` records the wire subset and conformance plan. The local actor model (11.1–11.5) is unchanged.

12. Ownership, linearity, and memory

12.1 Usage modes

A usage mode is a property of the type: `ExactlyOnce` for a linear resource, `AtMostOnce` for an affine reference, and `AnyNumberOfTimes` for an immutable image value. A linear type is a named set of traces; the end-of-trace rule is the mode's obligation. `linearity` declares the mode and `own`, `borrow`, `consume`, and `release` are the operations.

12.2 Reference linearity

`reclaimable` reports whether a reference is a reclaim candidate. `compact` is the opt-in compaction entry point; compaction preserves stable addresses and never moves a value whose identity is observable. Ownership state is execution state, not image data.

12.3 Epochs and promotion

An actor incarnation has an epoch. The scheduler records the outgoing turn's frontier growth, resumes the incoming epoch at the current frontier, and closes the epoch on termination. A value promoted at send or persistence is reclassified as durable without moving; an unpromoted value in a closed epoch is reclaimable when no live root reaches it. Reclamation reads every non-terminated actor's suspended state; a terminated actor is not a root.

12.4 The arena seam

`TensorHandle` is a stable storable reference; `TensorView` is a stack-only span projection. The managed arena allocates, resolves by generation, and invalidates handles on close. The evaluator is not routed through the arena; this is a host seam, not a second representation. Unmanaged storage is a later decision.

12.5 Frozen representation

The memory representation is frozen for the port: managed backing; shared immutable values with promotion at every escape; no copy-on-send; no evaluator arena routing. `Tensor` is not a `ref struct`; it cannot be stored in a class, a map, actor state, a mailbox, a closure, or persistence. `EntryIdentity` stays a struct. Struct and byref-like annotations are host performance choices, not semantics.

13. Errors and diagnostics

13.1 The taxonomy

`EvaluationError` is one union with typed fault families: `UnknownKind`, `ErrorSubject`, `ExpectedValue`, `ArityFault`, `ChannelFault`, `SortFault`, `LimitFault`, `EnumFault`, `ActorFault`, and the other named cases. A family case is written qualified to avoid union-case shadowing. `describeError` renders every case; `Violations.ofEvaluationError` yields the rule string for a source violation.

13.2 Diagnostics

A `Diagnostic` carries the error, the location, the baseline, and the position. A source violation adds a character offset when source text is available (D76). A failure at the kernel boundary is encoded as `(code, message)`; the typed error stays inside the kernel and the Prelude.

13.3 Refusals

A refusal MUST name the rule and the next action, as a DU case, not a string. `Result<_, string>` is a defect in new code. An undecidable case is reported as `Undecided(reason)`, never as a false; an unavailable resource is `Unavailable`, never an empty success.

The prose front-end returns `ProseError` cases (`ExpectedMatch`, `EmptyMatch`,

The taxonomy and the renderers are implemented (D94). Violation reporting covers the source, semantic, SMT-model, and unsat-core phases (D57, D61, D62). The prose front-end's refusals are named and tested (D142–D149). The shell effects `run` and `capture` are Stage 1a host words (D183); the `with` connective and the REPL are not implemented (D147).

The taxonomy and the renderers are implemented (D94). Violation reporting covers the source, semantic, SMT-model, and unsat-core phases (D57, D61, D62).

14. General IR and portability

14.1 Four representations

A program has four representations: surface text, the elaborated term list, the portable document, and the compilation target. The portable document is the interchange form; the lowered instruction stream is a compilation target, not a parallel runtime. The portable form is canonical JSON with a schema version; it carries no host objects.

The target runtime representation is the typed term list (D160): the program is a `Term list`, a quotation is a `Quote of Term list`, and `apply` splices its terms. P1 (D161) adds the core term shape

and its canonical portable document; P2 (D162) adds the kernel structural verifier, the opaque `VerifiedTermProgram` receipt, and the `scale_verify` entry at the current ABI version; P3a (D163) adds the token-to-term boundary; P3b (D165) makes the active program and caller frames `Term list` and executes numbers and words. Text quotations remain until P3c, code values are P3c, and stack typing is P4. The migration is staged in `docs/term-ir.typ`.

14.2 Documents

- the **pattern document**: schema version, form, kinds, predicates, clauses;
- the **expression document**: schema version, form, kinds, predicates, root;
- the **package document**: name, version, hash, imports, exports, capabilities, units.

Canonical constructor kinds and pinned named predicates make a document deterministic. A pattern template is a separate open form and is refused by the closed reader; `PatternElaboration` turns a template plus holes into a closed pattern.

14.3 Capabilities and trust

Capabilities are default-deny: a package declares what it needs and a host grants it. A host call is a named package/function request; a transport payload is a framed target plus bytes. The host adapter contract is the six-function adapter interface; an adapter contains no language semantics.

14.4 Source maps

A package carries unit-scoped source maps. A violation maps back to the unit, the location, and the character offset. A map that cannot resolve is a named failure, not a guessed position.

14.5 Status

The pattern IR and portable pattern document, the expression AST and its portable document, and the package/unit document are implemented (D96, D97). The general IR design is recorded; the full engine contract, the decision tree and unifier as portable components, and complete source maps are partial. The typed term list's P1 shape and portable document, P2 structural verifier, receipt, `scale_verify` entry, P3a token-to-term boundary, P3b typed control state, P3c quotation code values, structural history, and P3d text-channel removal are implemented (D160–D167). P4a–P4c add the type-inference core, expression inference, located diagnostics, and the lowering handoff (D173), but the parser does not yet emit `parseLocated` maps and the `logic`/`function` surface is not implemented. Stack typing and effect rows remain planned.

15. Kernel, ABI, and the image

15.1 The boundary

`components/kernel/scale_abi.h` is the specification and the artifact: it declares the C ABI and the callbacks. Anything the header declares is real; anything it does not declare does not exist to a host adapter. The kernel exports the ABI with `UnmanagedCallersOnly`; there is no C# export shim. The kernel has no Prelude state and no Prelude logic; a boundary fitness test enforces the type and reference separation.

15.2 Exports

ABI version 2 declares nine exports:

Export	Contract
`scale_init`	installs callbacks, validates the image, retains it
`scale_set_callbacks`	replaces the callback table for a live handle

Export	Contract
`scale_run`	resolves an exported unit and executes it
`scale_verify`	verifies a term-program document and writes its receipt
`scale_image`	imports or exports the retained image
`scale_host_call`	forwards a named package/function call
`scale_transport_send`	forwards a framed target and payload
`scale_free`	clears the handle and the retained image
`scale_abi_version`	returns the single ABI version

The exact set and the error codes are in the header; this table is a map, not a substitute.

15.3 The image

An image document is schema version 2: a form, kinds, predicates, units, groups, exports, imports, capabilities, primitives, and instructions. A unit has a name, a source hash, dependencies, explicit arity and capture slots, and a lowered instruction list. A group names its units, their initialization order, and each binding's classification. The loader validates names, arity, captures, groups, kinds, predicates, jumps, capabilities, dependencies, schema, and form. A call resolves through the unit's group, pops its arity, appends its captures from the caller's frame, and runs it; a closure invocation reuses its own frame. A short call or a missing capture is a named refusal. The image is data; the kernel validates it and retains it.

15.4 The primitive wire

A primitive call crosses the ABI as tensors: a declared primitive has an arity and a result count; the caller pops the operands, calls the host, and pushes the results. A primitive failure carries its named reason. A result-count mismatch is a protocol failure, never a silent truncation.

15.5 Invariants

No.	Invariant	Status
1	The kernel runs without the Prelude loaded	implemented
2	The Prelude is swappable data	implemented
3	The kernel state has no Prelude state	implemented
4	The kernel source contains no Prelude logic	implemented
5	Removing a feature requires no kernel change	partial
6	Adding a feature requires no kernel change	partial
7	Errors are attributed to the raising layer	partial
8	Upcalls are real	partial
9	Version equality is enforced at load time	partial

The full table with the enforcing test or `not yet` is `docs/INVARIANTS.typ`. D160 proposes a tenth invariant: the kernel executes only a term program with a valid verification receipt. P2 issues and binds the receipt through `scale_verify`; the execution refusal lands with the P3 runtime representation. The kernel verifier is the execution authority; the Prelude may reject more but cannot authorize a term list the kernel rejects.

16. The compiled subset

16.1 Lowering

`lowerExpression` is total over the closed `Expression` union: every constructor either lowers or is a named `LoweringRefusal`. Supported: literals, words, quotations, blocks, `if`, `call` over a word, `let`, `lambda`, and ground `pattern-match`. Refused: `Pin`, `Repeated`, `Shadowing`, guards, duplicate pattern variables, duplicate parameters, reserved binding names, calls to bound names, and the declaration/message nodes. A refusal is a lowering-time decision; the interpreter is the fallback.

`lowerExpression` returns a `LoweredProgram` with a unit table and entry instructions. A lambda becomes a named unit; `MakeClosure` carries its arity. `CallClosure` pops the arguments, refuses an underflow by name, and installs the frame in parameter order. Enclosing lexical names are captured by free-variable analysis (D133).

16.2 Slots, frames, and history

A lowered `let` writes a frame binding (for `lookup`) and a fresh slot (for a bare name). Slots are local to a unit; a unit prologue copies its frame into slots. The lowered instruction set has no step record and no resumable program counter. Compiled artifacts therefore exclude history, bookmarks, and rollback (D120); the interpreter remains the only history-recording engine.

16.3 The IL writer

The writer is a JIT-only backend behind `EnableWriter` (Debug only by default). The emitted method is `static KernelState<EvaluationState> Run(KernelState<EvaluationState>)`. It threads the real store, stack, roots, words, frames, and host, and calls the AOT-safe `WriterRuntime` for value instructions, stack access, dispatch, and failure. Value instructions share `LoweredAbstraction.step` with the interpreter. Slots are CLR locals. An unsupported instruction is a named refusal. Each `LoweredUnit` becomes a method taking `(KernelState<EvaluationState>, EntryIdentity[])`; a `MakeClosure` immediately followed by `CallClosure` compiles to a direct call, with `EnterUnit`/`ExitUnit` managing the frame and `PushCapture` reading it. A first-class closure is refused (D134).

The release publish passes `EnableWriter=false`, so the writer is unreachable from the AOT entry. A Release JIT run that needs the writer tests passes `-p:EnableWriter=true`.

16.4 Status

Lowering, lambda units, pattern compilation, and the writer harness are implemented for the supported subset. The writer emits one method per `LoweredUnit` and compiles direct lambda calls (D134). It still refuses first-class closures plus the tensor and pattern opcodes. The statically referenced AOT artifact comes after the remaining opcodes.

17. Prose sentences and the core language

17.1 The core language

The core language is Scale's own concatenative surface and general IR. Basic English and the prose surface are front ends over it; direct programmers use it directly. It is not the unified Julia/Elixir language: that direction is dropped (D151), and there is no shared source language, type hierarchy, or dispatch model between implementations. Different implementations are separate instances in their host languages and integrate through the actor model's IPC (11.7).

The Scale Prelude port is deferred until the core port is complete (D133); the F# reference remains the shipped Prelude.

17.2 Prose sentence grammar

```
sentence    ::= statement | transition
statement   ::= quantifier subject relative? verb "."
transition  ::= "when" subject ("that")? disjunction "," "every" subject verb "."
quantifier  ::= "every" | "no" | "some"
relative    ::= "that" disjunction
disjunction ::= conjunction ("or" conjunction)*
conjunction ::= condition ("and" condition)*
condition   ::= predicate
              | "is" field
              | "has" field comparison integer
predicate   ::= a FieldName whose surface form begins with "is "
verb        ::= a FunctionName
comparison  ::= "greaterThan" | "atLeast" | "lessThan" | "atMost" | "equalTo"
integer     ::= an optionally signed base-10 integer
```

The grammar is order-sensitive: the quantifier is sentence-initial, the subject is a type name, `that` introduces the optional relative clause, and the verb is final. A full stop is required. There is no implicit subject and no default verb. `and` binds tighter than `or`.

17.3 Quantifiers and transitions

`every` is universal affirmative; `no` is universal negative and emits a negated postcondition under the same preconditions; `some` is existential and emits a witness predicate with a satisfiability obligation. `all` is refused with `UnsupportedQuantifier`; use `every`.

A transition is event-triggered: `when` is sentence-initial, the trigger and the effect must name the same subject, and the outcome must be `every`. The trigger's conditions are the precondition and the verb is the effect, so the emitted property is `precondition ==> effect`.

17.4 Conditions

A condition is a boolean predicate phrase (`is alive`, canonicalized to `isAlive`), an explicit copula and field (`is health`), or a numeric comparison (`has health greaterThan 0`). A numeric comparison is a typed `NumericCondition(field, comparison, value)`; the SMT theory is integer-valued, so a decimal threshold is `Unavailable`, never rounded. A bare field noun is not a condition.

17.5 Refused surface

Articles, pronouns, `not`/`never`, `all`, and `when` away from the sentence start are refused with named parse errors. `or` is admitted between conditions. An unknown word, an inactive word, a missing full stop, a misplaced token, a missing comparison, a missing transition comma, and a mismatched transition subject each have a named case.

17.6 Status

Prose sentence grammar stage 1, transitions, and disjunction are implemented (D18, D55, D58). The grammar is intentionally a starting point, not a complete natural language. No expansion happens until a product sentence requires it (D131); the staged completion in `docs/prose-grammar.typ` is a map, not a commitment.

17.7 Prose program surface

The prose surface is a third front-end (D142–D149), documented in `docs/prose-surface.typ`.
A `.prose` file holds sentences that end with a full stop; keywords are lowercase, and no `end` closes a block.

A match has two forms. The inline prose form uses commas to delimit the multiple-choice construct: `match 5, when 1 then "one", otherwise "many".` The line form is the alternative: `match 5.` followed by `when ...` and `otherwise ...` lines. A line break does not start a new sentence, so the clause words stay lowercase. A comma outside the choice positions is `UnexpectedComma`; it never separates arguments.

Names are lowercase words separated by spaces, canonicalized through the vocabulary like the Prose sentence grammar: `is alive` is the surface for the `isAlive` token. A free multiword name is a `state of player` phrase. A `.prose` text literal may contain spaces and keeps its case. A nested match is closed by `close`; the line form writes `close.` and the inline form writes `close` where the enclosing sentence continues.

A program is a sequence of sentences. A one-sentence program keeps its expression AST; two or more become an `ExpressionAst.Block` in source order. The expression lowers with `EvaluatorLowering` and runs on the kernel `LoweredAbstraction` over one stack; the final stack top is the result. `scale --prose PATH` reads and runs a `.prose` file. The parser returns `Result<Expression, ProseError>` with named refusals. The shell effects `run` and `capture` are host words executed by `components/shell-host` (D183): `run` pushes the exit status, `capture` pushes standard output, and the script exits with the last command status. Both `run("echo" "hello")` and `run "echo" with "hello".` are accepted; the `with` connective is implemented (D187). A REPL is not implemented (D147).

Prose has no logic definitions (D184). A `.book` file contains definitions and compiles to a `PackageAst.PackageDocument`; a book with a `main` export is a spell and a book without one is a library. `scale --book` classifies and, for a spell, runs `main`; `scale --cast` calls an exported word. Prose calls a book with `cast("path" "word" args...)`, a host word handled by `components/book-host`. `logic` in a `.prose` file is `LogicOutsideBook`. A book is one recursive group: self, forward, and mutual calls are ordinary calls (D194). The optional `logic` and `cycle` markers are checked against the definition's book calls; `and` groups cycle definitions F#-style. A lowered word call that reaches a runtime lexical binding is a named refusal (`DynamicCall`), not a silent value push (D189).

18. Verification, refinement, and repair (extension)

18.1 SMT

Z3 is the verifier, in-process. `Microsoft.Z3` 4.12.2 is the package; `/usr/bin/z3` 4.16.0 is the offline fallback. The obligation is a typed predicate over the field sorts. `Status.UNKNOWN` is reported, never swallowed as a pass. A resource limit is `Unavailable`, not a proof.

18.2 Sorts and the lattice

The encoding carries Real, Integer, and Natural field sorts. Real is SMT Real and QF_LRA; Integer and Natural are Int and QF_LIA; a mixed domain is QF_LIRA. Natural is constrained to `>= 0`. The verification type lattice gives Z3, Hedgehog, and the refiner one coordinate system (D25). A guard threshold is validated against the field sort; the refiner uses a Real tolerance and clamps Natural at zero.

18.3 Refinement and repair

The refiner's closure is proved over the domain; witness-guided guards cover large domains; the repair search is minimum-edit and bounded. A violation reports the phase, the source, the semantic stage, the SMT model or unsat core, the minimum edit, and the presentation. A repair is a proposal, not a silent rewrite.

18.4 Decision tables

A decision table is a partition by comprehensions. The table surface, the emitter, the Prose sentence sink, and the verified artifact are implemented (D13, D73, D74). A table is proven where it is symbolic; it is not silently sampled.

18.5 Status

SMT, the type lattice, guard bridging, strictly verified emission, violation reporting P0–P2, the refiner, witness-guided guards, repair, and decision tables are implemented. Arbitrary core-language programs are not proved; verification is a feature of the specified/compiled forms, not of every program. Z3 verification is deferred while the language stabilizes (D131); the existing features remain, but new verification scope waits.

19. Persistence and interchange

Persistence serializes only durable or promoted entries across the live actor and every actor snapshot. The frontier and surviving addresses are unchanged, so the result is a sparse image. Execution state is never serialized. A reader MUST re-thread the association roots after loading. The image is append-only and the portable documents are canonical JSON; a host interchange uses the package document, not a memory dump. Persistence is implemented (D87); compaction beyond the initial store is an extension.

A store is a versioned, length-checked binary document. Semantic corruption and truncation are both `CorruptImageException` with a named `CorruptStore` reason. Tensor element counts are bounded by `maxTensorElements`; interned text is bounded by `maxTextBytes` before allocation. A reader never returns a partial store.

20. Versioning and conformance

20.1 One integer

The kernel, the Prelude image, and every host adapter share one monotonic integer version. It is not SemVer and has no semantic meaning: it says whether two artifacts belong to the same release. The current ABI version is 2; the image schema moved to 2 with unit groups (D192). The image schema is 2.

The number appears in the header (`SCALE_ABI_VERSION`), in the kernel's exported `scale_abi_version()`, and in every adapter's README. Compatibility is equality: an adapter refuses a kernel whose version differs; the kernel refuses an image whose schema differs. There is no partial compatibility and no matrix.

20.2 Release bundle

A release at version N includes the kernel, the Prelude image, and every adapter at N. Every release bumps every artifact. The cost is a coordinated rebuild; the benefit is that a mismatch is a deployment error, not a negotiation.

20.3 Conformance

The conformance test is a C harness that loads the kernel, initialises it with a known Prelude image, wires a callback table, runs a known program that exercises a kernel primitive and a Prelude word, verifies the result and the dispatch, checks the version, and destroys the handle. It is the executable definition of an adapter. It is not present yet because no adapter has landed; until then the SageFs suite is the acceptance gate and the invariants table records the gaps. The first target is a C adapter (D130).

The status matrix is enforced by ratchet tests: a row marked implemented or refused has a representative check, and changing a status changes the test (D135).

21. Status matrix

The suite at this state is 1530 example checks (1488 examples, 42 properties), 33 granular cases (11 snapshots, 13 integrations, 9 writer checks), and 91 discovered tests. D148 adds the lowering differential property, the prose-forms property, and the lowered-IR snapshot; D154 adds the actor IPC integration; D155 adds the capability and remote-supervision integration; D157 adds the language-level remote-send integration; D159 adds the wire golden-vector interop; D161 adds the term portable vectors; the boundary batch adds the malformed and forged-handle cases; D162 adds the P2 verifier, receipt, and `scale_verify` checks; D163 adds the P3a token boundary; D165 adds the P3b typed control-state checks; D166 adds the P3c code-value and structural-history checks; D167 adds the P3d text-refusal cleanup.

The table is generated from `docs/architecture.edn` by `tools/architecture-check.fsx`.

Feature	Status	Evidence / note
Tokenizer, literals, positions	Implemented	example suite; the prose lexer is separate (D142–D149)
Concatenative evaluator, stack, arithmetic	Implemented	example suite; differential corpus
Control: quotations, `if`, `apply`, `define`, `control`	Implemented	D21, D51
Frames, `lookup`, closures	Implemented	D45
`let`, lexical bare names	Implemented	D122; differential tests
`lambda` parameters, arity, capture	Implemented	D123, D133
Pattern matching, ground subjects	Implemented	D125
Pattern `Pin`/`Repeated`/`Shadowing`/ guards	Partial	interpreter implemented; lowered refusals; D130
Open-subject matching	Implemented	interpreter only
`PatternAnalysis`	Partial	best effort, not a proof
Declaration/message nodes	Refused in the lowered path	D130; interpreter-only until the registry contract
Sets, sorts, set operations	Implemented	D14, D36, D39
Subtyping (⊆ as a word)	Value-level set-subset	D131; type-level if a consumer
Sequences	Planned	D132; ordered projector and words
Recursive types	Planned	D132; recursive reference and fixpoint
Multisets	Out until a consumer	D132
Co-products	Out until a consumer	D132
Unification, occurs check, Miller patterns	Implemented	D15, D59

Feature	Status	Evidence / note
P4 type-inference core	Partial; core landed	D173, D174; P4a types/schemes/ constraints, P4b expression inference, and the single built-in word index in `PreludeSignatures` (`primitiveTable` is derived from it), exact effect sets, no rows or subtyping
P4 source ranges and lowering handoff	Partial; call ranges and CLI	D173; `InferenceDiagnostic`, `lowerExpressionWithInference`, `parseLocated` root and call/partial child ranges, `--prose-typecheck`; other child paths still open
`logic`/`function` surface	Partial	D177; block-scoped `logic` values and functions, bound closure calls; `function`, `close`-terminated bodies, currying, and recursion remain D172 design
F# package recursion classification	Implemented for the supported F# subset	D192, D194, D196, D197; F# abstraction, manifest tiers, diagnostics, metadata, package lowering, FCS read-off, supported body translation, classification metadata wire; unsupported forms refuse
Classes, slots, instances, precedence	Implemented	D43–D47
Method combination, metaclasses	Implemented	D48, D49
Conditions and restarts	Implemented	D50
Actors, mailboxes, links, monitors	Implemented	D63–D69
Supervision and deadlines	Implemented	D63
Channels and ChannelEvents	Implemented	D20, D26
Ownership, linearity, epochs, reclaim	Implemented	D30, D52–D54, D80–D90
Arena contract	Implemented as a seam	D89; evaluator not routed
Persistence of durable entries	Implemented	D87
Image loading, validation, swap	Implemented	D100, D105
C ABI, nine exports	Implemented	D95–D109, D162, D175, D192; `scale_verify` at ABI version 2; one independent handle per instance
Version equality at load	Implemented for the kernel and C adapter	D136, D175; ABI equality test, image-schema refusal, the adapter's compile-time guard, and the native kernel gate (D180)
Adapter conformance test	Implemented; runs in alpha against the	D136, D180; `just kernel-aot` publishes the kernel and runs

Feature	Status	Evidence / note
	NativeAOT kernel	`components/adapter-c/conformance/`
Expression lowering, total	Implemented	D119, D148 P1; named refusals and the lowering differential
Lambda units, pattern compiler	Implemented	D123, D125, D133
IL writer, units and direct calls	Partial	D128, D130, D134; first-class closures refuse
Statically referenced AOT artifact	Planned	after the remaining opcodes
Portable pattern/expression/package documents	Implemented	D96, D97
General engine contract, source maps	Partial	D97
Prose sentence grammar stage 1	Implemented	D18, D55, D58
Prose surface, first slice	Implemented	D142–D149, D177; 36 checks, D148 P2, `scale --prose PATH`
Prose shell effects	Implemented, Stage 1a/1b	D147, D183, D187; `run`/`capture` host words, `shell-host`, last-command status, the `with` connective
Books and spells	Implemented, Stage A	D184, D194; `.book` packages, `--book`, `--cast`, prose `cast`, `book-host`, automatic recursive groups and markers
Callable contract	Implemented, first slice	D189; strict lowered `CallWord`, `DynamicCall`, book target `Apply`
Prose sentence grammar, full natural coverage	Deferred	D131; only on a product need
SMT, lattice, refiner, repair	Implemented; deferred	D10, D25, D56–D62, D91, D92; D131
Decision tables	Implemented	D13, D73, D74
DSPy-shaped programs	Extension, prepared	D42
BEAM operational parity, unmanaged arena	Optional, not on the path	D116
Implementation-instance IPC	Partial; S1/S2	D151, D154–D156; numeric envelope, capability 1 worker, monitor, down, Elixir wire harness
Language-level remote send	Partial; one-way	D157; `send-to` records a transport request, host drains
F#-equivalent pipeline	Implemented	D168–D171; LastArgument/ FirstArgument, parser, portable document, lowering, whitespace stage arguments, explicit _ partial application, reverse pipe; automatic currying deferred

Feature	Status	Evidence / note
Typed term IR / code as data	Partial; P3d	D160–D167; core term shape, portable vectors, structural verifier, receipt, `scale_verify`, term program, tagged quotation code values, structural history, no text quotation fallback; surface clause parsing and the kernel bridge keep token projections
Actor wire protocol	Partial	D152–D159; single-segment codec with send/reply/monitor operations; reference conformance for the frozen subset
Wire golden vectors and reference interop	Implemented for the frozen subset	D159; `capnp` 1.5.0 and checked-in vectors
Unified Julia/Elixir language	Rejected	D93 revoked by D151; `docs/rejected-ideas/unified-language.typ`
Scale Prelude port	Deferred	D133; after the core port
Native port P1	Implemented for the landed subset	D98–D125
Native port P2	Partial	D126, D128, D130, D134
Polylith workspace	Implemented	D137–D139; `just poly-check`
Nu external node and host skeleton	Partial	D140, D181; engine vendored, host skeleton; plan in `docs/nu-gaia-imsim.typ`
Demo content root	Implemented	D141; checkout and publish payload
Unit groups and group call frames	Implemented for function groups; value-unit initialization planned	D192, D193, D194; Unit/UnitGroup, arity, captures, InitOrder, classification, call/closure split, depth and arity refusals, book groups
Logic and cycle definitions	Implemented	D195; core logic/cycle words, plain-name recursion, and groups, logic self-call refusal
Book package contract	Implemented	D198, D199; lazy load, per-unit hashes and dependencies, main arity, call arity, cache, open imports and capability union; external package imports deferred
Shell policy sandbox	Implemented for programs, write roots, and environment	R12; allowed programs, allowed write roots, optional cleared environment; read paths, network policy, and package signatures remain

Feature	Status	Evidence / note
Semantic test matrix	Recorded	R13; call graphs, effect timing, capabilities, persistence, AOT, fuzz and property dimensions

22. References

- `docs/INVARIANTS.typ` — the constitution and the enforcement table.
- `docs/decision-records.typ` — D1–D199, the decisions behind this text.
- `docs/term-ir.typ` — the typed term list, the kernel/Prelude split, and the migration.
- `docs/commitments-ledger.typ` — the live status and the counts.
- `docs/roadmap.typ` — the current work pointers; `docs/roadmap-archive.typ` is the completed sequence.
- `docs/actor-ipc.typ` — implementation instances and the actor IPC staging.
- `docs/binary-format.typ` — the actor wire protocol, its subset, and its conformance plan.
- `docs/rejected-ideas/unified-language.typ` — the dropped unified Julia/Elixir proposal.
- `docs/set-theoretic-typing.typ`, `docs/type-lattice.typ` — sorts and the dial.
- `docs/cell-system.typ` — classes, dispatch, combination, metaclasses.
- `docs/memory-model.typ` — image, handles, views, epochs.
- `docs/general-ir.typ` — the portable IR and the adapter contract.
- `docs/kernel-inventory.typ` — the kernel boundary and the risks.
- `docs/native-dotnet-port.typ` — the port plan and the phase acceptance.
- `docs/prose-grammar.typ` — the controlled grammar.
- `docs/nativeaot.typ` — the AOT contract.
- `components/kernel/scale_abi.h` — the authoritative C ABI.